Capstone Final Reflection

Group 7 - LM Compass
COMPSCI 4ZP6
Dr. Mehdi Moradi - Dr. Angela Zavaleta Bernuy
April 7, 2026

Sohaib Ahmed	ahmes127@mcmaster.ca	Coordinator/Researcher
Madhav Kalia	kaliam1@mcmaster.ca	Backend Engineer
Aadi Sanghani	sanghana@mcamster.ca	Backend Engineer
Gulkaran Singh	singg36@mcmaster.ca	Fullstack Engineer
Rochan Muralitharan	muralr3@mcmaster.ca	Fullstack Engineer
Owen Jackson	jackso4@mcmaster.ca	AI Engineer
Aryan Suvarna	suvarnaa@mcmaster.ca	Frontend Engineer

Links:
1. Application: lm-compass.com
2. GitHub Repository: github.com/LMCompass/LM-Compass

Table of Contents

1. Background
2. Problem Statement
3. Our Solution
4. Project Complexity
   - 4.1. Why is it complex?
   - 4.2. Technical Challenges
5. Achievements
6. Requirements Completion
7. Highlights & Lowlights
8. What we learned
9. References
10. Appendix

1. Background

Generative AI is expanding rapidly, making it increasingly difficult to evaluate the quality of responses from various Large and Small Language Models (LLMs/SLMs). Current evaluation methods rely mostly on subjective measures or narrow benchmarks, which limits fair comparison and practical adoption in the industry.

2 Problem Statement

Users and researchers currently lack a scalable tool to objectively evaluate and compare how different models respond to the exact same query or dataset. Consequently, it is difficult to reliably determine which specific model is best suited for a given task.

3. Our Solution

LM Compass is a web-based platform designed to evaluate and rank the quality of responses to queries from multiple Large and Small Language Models (LLMs/SLMs) using a scalable LLM-as-a-judge cross-evaluation system. By integrating user customizable judging rubrics and a variety of evaluation methods, we built an open and unbiased framework for researchers and developers to determine which models are best suited for specific queries or large sets of data.

4. Project Complexity

4.1 Why is it complex?

The core complexity of LM Compass stems from designing an LLM-as-a-judge cross-evaluation engine capable of concurrent multi-model routing. In our system, up to 4 models queried via OpenRouter must dynamically judge each other's candidate responses in a peer-review matrix. Building a reliable LLM-as-a-Judge system is inherently complex because language models are susceptible to intrinsic scoring biases (e.g., rubric order bias, recency bias)[9] and often struggle to consistently adhere to highly detailed task evaluation instructions [7].

Furthermore, mitigating these biases while managing subjective disagreements necessitated the integration of “Rationale Based Self-Critique Loop” (RL4F) and "Human-in-the-Loop" (HITL) evaluation methods. As highlighted in recent literature, integrating human feedback to refine model outputs or utilizing human insights to dynamically adapt rubric-based autograding significantly elevates the architectural complexity of standard AI evaluators. Though LLMs are highly versatile for text analysis and short-answer autograding [10], achieving a fair, automated consensus across diverse models remains a massive computational and theoretical challenge.

To rigorously test our evaluation engine, we benchmarked the system using 250 randomized questions from the Humanities Last Exam (HLE) dataset [8] by comparing LM Compass’ Experiments. The HLE is a highly complex set of questions representing expert-level human knowledge, designed specifically to challenge automated grading systems across diverse academic subjects.

4.2 Technical Challenges

The development of the LM Compass web application involves several complex technical challenges, particularly in creating a reliable, high-performance environment for automated LLM-as-a-judge evaluation.

1. **Designing the evaluation methods**
   The primary challenge in designing evaluation methods was moving beyond subjective benchmarks toward a scalable, research-backed "peer-review" consensus-based ranking. While inspired by current research, the implementation required engineering a system that could balance computational cost with strong alignment to humans. The team specifically selected Prompt-based Rubric Scoring[9], Human-in-the-Loop (HITL) Adaptive Rubrics[2], and Rationale-Based Self-Critique Loops (RL4F)[1] because research identifies these as the most effective for maximizing evaluation accuracy while minimizing the need for expensive manual oversight. Implementing HITL in particular was challenging because we had to create a new framework where the AI can pose targeted questions to a human for clarifying questions about the rubric. Also, to handle long completion times, we had to handle expiring JWTs to save conversations. One of the key concerns of our app that we had to address was how to mitigate bias. Judge models may inherently favour their own outputs so to counter this, our platform ensures we anonymize responses during the judging phase and do not allow models to judge their own responses.
2. **Concurrent Multi-Model Querying and Efficient Streaming**
   Interacting with up to 4 LLMs simultaneously via OpenRouter introduced significant performance and state management hurdles. In the backend, we utilize *Promise.allSettled* to execute concurrent API requests, ensuring that one failing or slow model does not block the entire evaluation pipeline. To maintain a responsive user experience, we implemented Server-Sent Events (SSE) to stream progress phases (such as "Querying models" and "Evaluating responses") to the UI in real-time. However, the total latency remains bounded by the slowest model in the batch and is further impacted by the OpenRouter resource cap, which can throttle high volumes of parallel requests.
3. **Parsing and Handling Unpredictable LLM output**
   Automating the "LLM-as-a-judge" workflow requires extracting structured data from models that often return inconsistent or malformed JSON. The system handles this through a specialized *extractScoreAndReasoning* method that uses brace counting and markdown code block parsing for robust extraction of scores and reasoning. When a judge model returns non-JSON text or times out, the engine is designed to assign null scores or default values rather than crashing. This allows the rest of the consensus to proceed. This stability is maintained through strict type validation and error boundaries that prevent network failures or malformed outputs from leaving the UI in an inconsistent state.

5. Achievements

We consider three main achievements of our system:

1. **Application uniqueness**: Unlike standard or comparable benchmarking applications (e.g., LM Arena) that rely on subjective human opinion to rank the quality of model responses, LM Compass uniquely features a peer-review consensus-based ranking. During this cross-evaluation, candidate responses are anonymized so models cannot favor their own outputs, minimizing inherent biases and delivering a truly objective evaluation.
2. **Topline Metric**: During our benchmark against 250 randomized queries from the HLE dataset, LM Compass consistently outperformed the highest-performing standalone models in evaluation accuracy (see Figure 1). We also found that in a given experiment, no single model won more

than 22% of the total test cases (see Figure 2). This shows that model selection must be task-dependent, validating the core benefit and necessity of our consensus system.

3. **Stability Despite Variance**: Integrating rank correlation metrics like Kendall's Tau-B_4 highlights the inherent complexity of LLM-as-a-judge systems by mathematically quantifying how frequently different models disagree when evaluating the same responses. Navigating this high degree of variance is a major achievement for LM Compass. Even when judge agreement drops to a low score, the platform still successfully computes a stable, peer-reviewed consensus that effectively filters out individual model biases and hallucinations.

6. Requirements Completion

Requirement Level	Requirement Description	% Completion	Notes
P0	Submit Query: Frontend UI to submit free-text prompts and backend payload validation.	100%	Handled via PromptInput component
P0	Model API Key Vault & Routing: Secure storage of API keys and OpenRouter integration.	100%	Implemented via our encryption module and OpenRouter integration
P0	Candidate Generation: Concurrent multi-model querying and UI streaming.	100%	Achieved utilizing Promise.allSettled and SSE streaming
P0	Judging & Scoring Engine: LLM-as-a-judge system to evaluate and pick a consensus winning response.	100%	Implemented PromptBasedScoring (n & n^2 variations), RL4F, and HITL evaluation methods.
P1	User input on non-consensus: Manual override for tie scenarios.	100%	Users can review responses & select a winning response.
P1	Rubric Management: CRUD UI for custom evaluation criteria.	100%	Users can create, review, edit, delete, and select rubrics dynamically.
P1	Query and Result History: Save chat history and evaluation records to the database.	100%	Implemented by only saving winning responses to the database, avoiding clutter from large evaluations.
P1	Data Experiments: Dashboard to import datasets (CSV/Parquet) for batch evaluation runs across models & analytics for completed runs which can be exported.	100%	Users can run large-scale batch experiments for datasets. Individual rows do not persist in the database, and results can be exported to

			a PDF.

7. Highlights & Lowlights

Highlights**:**

- **Secure Architecture and API Management**: Designed a secure backend to handle sensitive user data and external API access. We built a custom encryption module to ensure the personal OpenRouter API keys that LM Compass requires users to provide to query multiple models are encrypted before being stored in Supabase. Furthermore, we enforced Row Level Security (RLS) policies to ensure that user chat histories, custom rubrics, and evaluation data are isolated and accessible only to their authenticated owners.
- **Backend Bias Engineering and Resiliency**: Able to avoid well-known biases of LLM-as-a-judge frameworks by successfully implementing mitigation strategies. The system anonymizes responses during cross-evaluation to ensure that models never judge their own response. Also, we built resilient logic to handle invalid JSON output from models by assigning null values to ensure that the scoring engine is stable at all times and doesn't crash.
- **Applied Research and Feature Engineering**: We built tools for researchers that include custom rubric management and an experiment dashboard for running large-scale batch experiments across datasets. Also, we were able to integrate complex academic research into practical features, such as implementing different evaluation methods:
    - Prompt-Based Scoring: Scores every other model's responses based on your defined grading criteria.
    - Human-In-The-Loop: Detects ambiguous cases in the grading criteria, and prompts the user with targeted questions to clarify and update the criteria.
    - Rationale-Based Self-Critique Loops: Critique and revise the logic of model responses over one or more refinement iterations.

Lowlights**:**

- **External API Bottlenecks and Cost:** Having to rely on external model providers through OpenRouter creates latency challenges. Since evaluation time depends on the slowest model and having large concurrent requests can slow down the system response time. Since multiple models are being queried, costs will naturally be higher than using a single-model system like ChatGPT.
- **Rubric sensitivity:** Outcomes depend heavily on rubric wording and weighting; small rubric changes can shift winners, which complicates reproducibility and comparisons across experiments.

8. What we learned

In developing LM Compass, we extensively learned about the LLM-as-a-judge research topic and the immense work that is being put into it. One of the major takeaways was learning about the diverse evaluation methods used to ensure fairness and accuracy. For instance, we learned about Prompt-Based Scoring, where models score each other based on pre-defined criteria. Testing with different rubrics taught us how critical prompt design is to mitigate inherent scoring biases within models. Through our discovery of Rationale-Based Self-Critique Loops, we gained insight into how models can iteratively critique and revise the logic of their responses over multiple cycles to improve output quality. Finally, we learned about the important value of Human-in-the-Loop feedback. By designing our system to detect ambiguous cases, prompt users to clarify grading criteria and allow manual overrides, we realized that involving human oversight is essential to dynamically refining rubrics and maintaining control over the evaluation process.

9. References

Below is a list of references cited in this reflection and used throughout our ideation, research, and development of LM Compass.

1. Akyürek, A. F., Akyürek, E., Madaan, A., Kalyan, A., Clark, P., Wijaya, D., & Tandon, N. (2023, July 11). *RL4F: Generating natural language feedback with reinforcement learning for repairing model outputs*. arXiv.org. https://arxiv.org/abs/2305.08844

2. Chu, Y., Li, H., Yang, K., Copur-Gencturk, Y., & Tang, J. (2025, December 1). *LLM-based automated grading with human-in-the-loop*. arXiv.org. https://arxiv.org/abs/2504.05239

3. Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., & Guo, J. (2025, October 19). *A survey on LLM-as-a-judge*. arXiv.org. https://arxiv.org/abs/2411.15594

4. Landesberg, E. (2026, March 12). *When LLM judge Scores Look good but best-of-N decisions fail*. arXiv.org. https://arxiv.org/abs/2603.12520

5. Li, H., Dong, Q., Chen, J., Su, H., Zhou, Y., Ai, Q., Ye, Z., & Liu, Y. (2024, December 10). *LLMS-as-judges: A comprehensive survey on LLM-based Evaluation Methods*. arXiv.org. https://arxiv.org/abs/2412.05579

6. Li, Q., Dou, S., Shao, K., Chen, C., & Hu, H. (2026, February 3). *Evaluating scoring bias in LLM-as-a-judge*. arXiv.org. https://arxiv.org/abs/2506.22316

7. Murugadoss, B., Poelitz, C., Drosos, I., Le, V., McKenna, N., Negreanu, C. S., Parnin, C., & Sarkar, A. (n.d.). *Evaluating the evaluator: Measuring Llms' adherence to Task Evaluation Instructions*. Proceedings of the AAAI Conference on Artificial Intelligence. https://ojs.aaai.org/index.php/AAAI/article/view/34157

8. Safety, C. for A. (n.d.). *Cais/Hle · datasets at hugging face*. cais/hle · Datasets at Hugging Face. https://huggingface.co/datasets/cais/hle

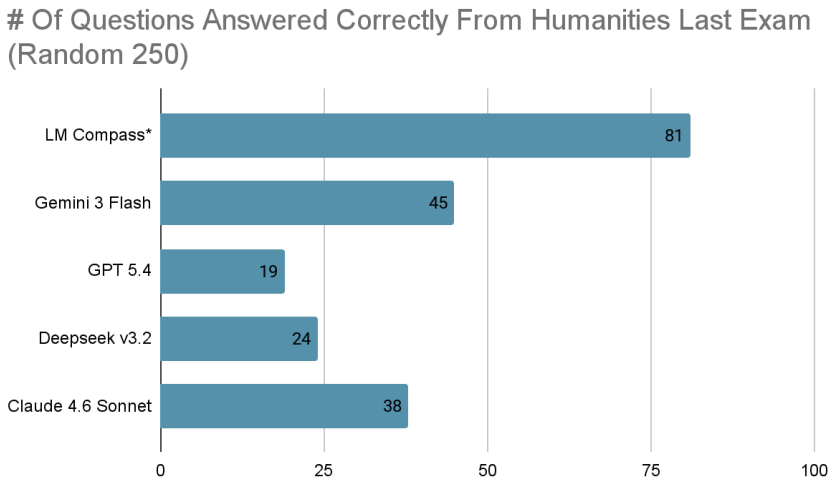
9. Schneider, J., Schenk, B., & Niklaus, C. (2024, July 8). *Towards LLM-based Autograding for short textual answers*. arXiv.org. https://arxiv.org/abs/2309.11508

10. Törnberg, P. (2023, July 24). *How to use LLMS for text analysis*. arXiv.org. https://arxiv.org/abs/2307.13106

11. Yang, H., Li, M., Zhou, H., Xiao, Y., Fang, Q., & Zhang, R. (2023, December 24). *One LLM is not enough: Harnessing the power of Ensemble Learning for Medical Question Answering*. medRxiv : the preprint server for health sciences. https://pmc.ncbi.nlm.nih.gov/articles/PMC10775333/

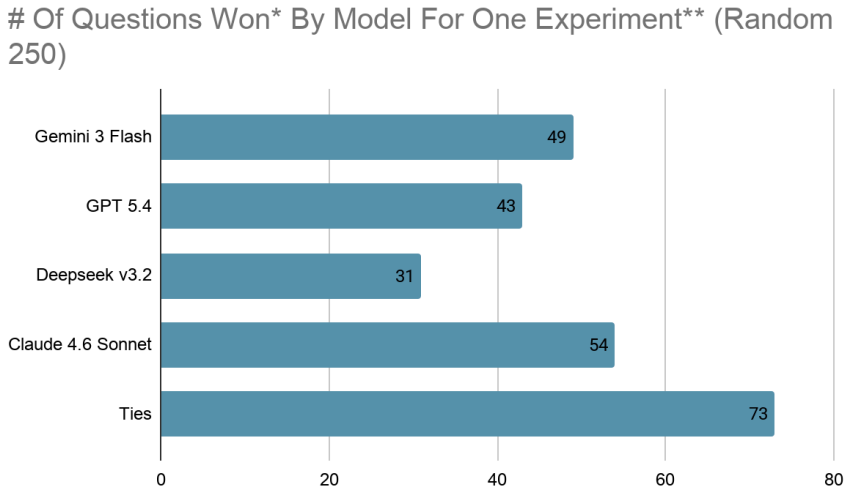
Appendix

Figure 1



*LM Compass Experiment using One-shot Prompt Based Scoring with Gemini 3 Flash, GPT 5.4, Deepseek v3.2, Claude 4.6 Sonnet

Figure 2



*Evaluated winners, not necessarily correct answers
**LM Compass Experiment using One-shot Prompt Based Scoring with Gemini 3 Flash, GPT 5.4, Deepseek v3.2, Claude 4.6 Sonnet